



GUIA DEFINITIVO: DO CÓDIGO MONOLÍTICO À ARQUITETURA EM CAMADAS

Construindo um Sistema Robusto com TypeScript, Express, Zod, Multer e SQLite

Coleção: Engenharia de Software Aplicada à Web

Disciplina: Programação para Internet II – IFPI

Autor: Prof. Sênior de Programação Web & Arquitetura de Sistemas

Versão da Obra: 1.0.0 – Edição Definitiva para Estudos

Público-Alvo: Estudantes de Tecnologia, Desenvolvedores Web e Engenheiros de Software em
Formação

*"Qualquer tolo pode escrever código que um computador entende. Bons programadores escrevem
código que humanos podem entender."*

— Martin Fowler

1.	PREFÁCIO DO PROFESSOR
°	1.1. A Jornada do Estudante: Do "Código que Funciona" ao "Código Profissional"
°	1.2. O Domínio do Mini-Prontuário: Inspiração no Padrão Internacional HL7 FHIR
°	1.3. O Método dos 14 Passos (O Roteiro de Refatoração)
2.	MAPA DA ARQUITETURA E VISÃO SISTÊMICA
°	2.1. O Problema do Código Monolítico (<i>Flat Server</i>)
°	2.2. A Solução: Arquitetura em Camadas (Separation of Concerns - SoC)
°	2.3. Diagrama Completo de Fluxo de Dados (Ponta a Ponta)
°	2.4. A Regra de Ouro do Desacoplamento
3.	ECOSSISTEMA & CONFIGURAÇÃO DO AMBIENTE
°	3.1. <code>package.json</code> : Escolhas Tecnológicas e Dependências
°	3.2. <code>tsconfig.json</code> : Rigor de Tipagem e TypeScript Moderno
°	3.3. Scripts de Automação: <code>dev</code> , <code>db:reset</code> , <code>check</code>
4.	A CAMADA DE PERSISTÊNCIA & BANCO DE DADOS RELACIONAL
°	4.1. <code>database/schema.sql</code> : Modelagem Relacional, Tipos SQLite, Chaves Estrangeiras e Índices
°	4.2. <code>database/seed.sql</code> : Dados Fictícios e a Ética Médica de Dados Sintéticos
°	4.3. <code>scripts/reset-db.ts</code> : A Importância do Estado Limpo e Reprodutível
°	4.4. <code>src/db/database.ts</code> : Instanciação do SQLite e o Segredo do <code>foreign_keys = ON</code>
5.	A CAMADA DE NEGÓCIO: OS SERVICES
°	5.1. O que é um Service e Por que ele NÃO conhece HTTP?
°	5.2. <code>src/services/patients.service.ts</code> : Análise Detalhada Linha por Linha
°	5.3. Mappers: Convertendo <code>PatientRow</code> (SQL snake_case) para <code>Patient</code> (JavaScript camelCase)
°	5.4. <code>src/services/encounters.service.ts</code> : Atendimento Clínico e Integridade de Chave
6.	A CAMADA DE TRADUÇÃO & ORQUESTRAÇÃO: OS CONTROLLERS
°	6.1. O Papel Sagrado do Controller
°	6.2. <code>src/controllers/patients.controller.ts</code> : Extração, Chamada e Resposta HTTP
°	6.3. <code>src/controllers/encounters.controller.ts</code> : A Passagem Limpa de Identificadores
7.	A CAMADA DE ENTRADA: AS ROTAS MODULARES
°	7.1. O Recurso <code>express.Router()</code>
°	7.2. <code>src/routes/patients.routes.ts</code> : Composição de Rotas e Pipeline de Middlewares
	<pre>{ mergeParams: true }</pre>
8.	TRATAMENTO ROBUSTO DE ERROS: HIERARQUIA E MIDDLEWARE CENTRAL
°	8.1. A Morte do <code>try/catch</code> Repetitivo e dos Códigos Espalhados
°	8.2. <code>src/errors/HttpError.ts</code> : A Hierarquia Polimórfica de Erros HTTP (400, 404, 409, 413, 422)
°	8.3. <code>src/middlewares/errorHandler.ts</code> : O Middleware de 4 Parâmetros do Express
9.	INTEGRIDADE DE DADOS: VALIDAÇÃO DECLARATIVA COM ZOD
°	9.1. Validação Manual Imperativa vs Schemas Declarativos
°	9.2. <code>src/validation/patients.schemas.ts</code> : Construindo Schemas com Regexp e Mensagens Amigáveis
°	9.3. <code>src/middlewares/validate.ts</code> : O Middleware Genérico Interceptador e Sanitizador
10.	UPLOAD SEGURO DE ARQUIVOS COM MULTER
°	10.1. Riscos de Segurança na Web: Invasão, Path Traversal e Ataques DoS
°	10.2. <code>src/middlewares/upload.ts</code> : <code>diskStorage</code> , <code>UUID</code> Criptográfico e Filtro MIME
°	10.3. Servindo Arquivos Estáticos com Segurança em <code>/uploads</code>
11.	A INTEGRAÇÃO COMPLETA: O NOVO <code>server.ts</code>
°	11.1. O Antes vs O Depois: Anatomia de um <code>server.ts</code> Elegante
°	11.2. A Ordem Crítica de Registro de Middlewares no Express
12.	O FRONTEND MODULAR E REATIVO (VANILLA JAVASCRIPT)
°	12.1. Uma Arquitetura Limpa no Navegador Sem Frameworks Pesados
°	12.2. <code>public/index.html</code> : Semântica, Acessibilidade e Estrutura Bootstrap
°	12.3. Design System & CSS: <code>tokens.css</code> e <code>base.css</code>
°	12.4. <code>public/js/state.js</code> : Padrão <i>Single Source of Truth</i>
°	12.5. <code>public/js/api.js</code> : Camada de Comunicação e <code>FormData</code>
°	12.6. <code>public/js/errors.js</code> : Apresentação Consistente do Contrato de Erro
°	12.7. <code>public/js/render.js</code> : Renderização Declarativa Baseada em Estado
°	12.8. <code>public/js/app.js</code> : Orquestração Geral e Event Listeners
13.	LABORATÓRIO DE TESTES HTTP & AUDITORIA
°	13.1. <code>requests.http</code> : O Test Harness Declarativo
°	13.2. Cenários de Sucesso vs Cenários de Falha Proposital
14.	GUIA DE BOLSO DO DESENVOLVEDOR & CHECKLIST DE PROVA
°	14.1. Tabela Periódica dos Códigos de Status HTTP
°	14.2. Armadilhas Comuns e Como Não Cair Nelas
°	14.3. Checklist para Avaliação Prática e Projetos Reais

PREFÁCIO DO PROFESSOR

Querido(a) estudante,

Seja muito bem-vindo(a) a este livro-guia. Se você está cursando **Programação para Internet II** no **Instituto Federal do Piauí (IFPI)** ou simplesmente deseja elevar seu patamar de desenvolvedor júnior para um desenvolvedor pleno consciente, você tem em mãos o mapa completo do tesouro da arquitetura de software web.

Quando começamos a estudar programação web com Node.js e Express, nosso primeiro impulso é colocar tudo em um único arquivo: criamos o servidor, abrimos a conexão com o banco de dados, escrevemos queries SQL dentro da rota, validamos se a string está vazia com um monte de `if/else`, tratamos o erro e já enviamos a resposta HTTP ali mesmo.

Isso funciona? **Sim.**

Isso é aceitável em um ambiente profissional ou em um sistema real? **Definitivamente não.**

O código inicial deste projeto, batizado carinhosamente de **Mini-Prontuário**, começou exatamente assim: um único arquivo `src/server.ts` contendo mais de 250 linhas, onde banco de dados, roteamento, lógica de negócio e formatação HTTP estavam misturados como um macarrão espaguete.

Neste ebook, vamos dissecar o projeto **Mini-Prontuário**, examinando **cada arquivo, cada função, cada linha de código e cada decisão arquitetural**. Vamos transformar aquele servidor confuso em uma obra de engenharia modular dividida em camadas límpidas:

1. **Routes (Rotas):** Definem os caminhos e verbos.
2. **Middlewares:** Interceptam requisições para validar esquemas (Zod), processar uploads (Multer) e capturar erros globalmente.
3. **Controllers (Controladores):** Traduzem a linguagem do protocolo HTTP para o domínio da aplicação.
4. **Services (Serviços):** Guardam as regras de negócio puras e o acesso aos dados.
5. **Database (Banco de Dados):** Onde o SQL relacional garante a integridade dos registros.

Além disso, exploraremos a rica interface web construída em **Vanilla JavaScript modular**, provando que é possível ter reatividade, desacoplamento e excelente experiência de usuário sem a complexidade desnecessária de frameworks pesados.

Coloque um café ao seu lado, abra seu terminal e prepare-se para uma leitura aprofundada, instigante e definitiva.

CAPÍTULO 1: MAPA DA ARQUITETURA E VISÃO SISTÊMICA

1.1. O Problema do Código Monolítico (*Flat Server*)

Imagine que você foi contratado para manter um hospital e todos os médicos, enfermeiros, recepcionistas, farmacêuticos e o setor de limpeza ficam na mesma sala minúscula. O recepcionista tenta cadastrar um paciente enquanto o cirurgião está operando ao lado dele, e a secretária está arquivando prontuários na mesma mesa onde se manipulam remédios. O caos é garantido.

No desenvolvimento de software, chamamos isso de **Anti-padrão Monólito Flat** ou **God File** (Arquivo Deus). No início do nosso projeto, o arquivo `src/server.ts` era esse arquivo onipotente. Ele fazia:

- Leitura e configuração do servidor Express;
- Conexão e execução de queries SQL (`db.prepare`);
- Mapeamento de endpoints HTTP (`app.get` , `app.post`);
- Validação manual de strings, expressões regulares de data;
- Retornos de erros repetitivos (`res.status(400).json(...)` , `res.status(404)...`);
- Lógica de domínio (ex: verificar se o CNS do paciente já existe).

Quais são os sintomas clássicos de um código flat?

1. **Acoplamento Extremo:** Se você quiser mudar o banco de dados de SQLite para PostgreSQL, você precisa mexer dentro da rota HTTP.
 2. **Impossibilidade de Testes Unitários:** Como você testa a regra de "não permitir CNS repetido" sem subir um servidor HTTP inteiro?
 3. **Duplicação de Código:** A cada novo endpoint, você reescreve blocos de tratamento de erros idênticos.
 4. **Violação do Princípio da Responsabilidade Única (Single Responsibility Principle - SRP):** Uma função possui dezenas de motivos para mudar.
-

1.2. A Solução: Arquitetura em Camadas (Layered Architecture)

A **Arquitetura em Camadas** divide o sistema horizontalmente, onde cada camada possui uma responsabilidade estrita e conversa apenas com quem está imediatamente adjacente a ela.

[CLIENTE: Navegador / requests.http]



▼ (Requisição HTTP: GET, POST, etc.)

1. CAMADA DE ROTAS (src/routes/)

Mapeia URL + Verbo HTTP para o Controller correto



▼ (Passa por Middlewares)

2. MIDDLEWARES (src/middlewares/)

- validate.ts (Zod): Valida e sanitiza o payload
- upload.ts (Multer): Salva fotos e filtra tipo MIME



▼ (Dados válidos e prontos)

3. CAMADA DE CONTROLLERS (src/controllers/)

- Extrai parâmetros (req.params, req.body, req.file)
- Invoca o Service apropriado
- Formata o status HTTP (200, 201) e devolve o JSON



▼ (Chamada com parâmetros puros de domínio)

4. CAMADA DE SERVICES (src/services/)

- Regra de Negócio Pura (ex: "CNS já existe?")
- NÃO CONHECE Express, req, res
- Lança HttpErrors caso regras sejam violadas



▼ (SQL puro / Prepared Statements)

5. CAMADA DE PERSISTÊNCIA (src/db/ & database/)

- SQLite (better-sqlite3)
- Tabelas: patients, encounters



▼ (Caso ocorra qualquer erro em qualquer lugar)



CAMADA GLOBAL DE ERROS: errorHandler.ts

Captura exceções e gera resposta JSON padronizada

A Regra de Ouro do Desacoplamento:

"O Service não sabe que o HTTP existe."

Se amanhã decidirmos remover o Express e transformar essa aplicação em um assistente de linha de comando (CLI) ou conectar a um bot de Telegram, os arquivos em `src/services/` e `src/db/` não sofrerão uma única linha de alteração. Esse é o ápice do design de software profissional!

CAPÍTULO 2: ECOSSISTEMA & CONFIGURAÇÃO DO AMBIENTE

Vamos examinar a base operacional do projeto: os arquivos de configuração que ditam como o código é compilado, executado e gerenciado.

2.1. O Arquivo `package.json`

O arquivo `package.json` é o documento de identidade do projeto Node.js.

```
{
  "name": "mini-prontuario-camadas",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "Projeto-fio - Topico 2, etapa avancada (arquitetura em camadas, erros,
validacao, upload) - Programacao para Internet II (IFPI)",
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "db:reset": "tsx scripts/reset-db.ts",
    "check": "tsc --noEmit"
  },
  "dependencies": {
    "better-sqlite3": "^11.5.0",
    "express": "^5.0.1",
    "multer": "^1.4.5-lts.1",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/better-sqlite3": "^7.6.11",
    "@types/express": "^5.0.0",
    "@types/multer": "^1.4.12",
    "@types/node": "^22.9.0",
    "tsx": "^4.19.2",
    "typescript": "^5.7.2"
  }
}
```

Dissecando cada chave do `package.json`:

- `"type": "module"`: Define que o Node.js deve tratar todos os arquivos JavaScript/TypeScript como **ECMAScript Modules (ESM)**. Isso significa que usamos `import / export` nativos em vez do antigo `require() / module.exports` do CommonJS.
- `better-sqlite3`: O driver de banco de dados SQLite mais rápido do ecossistema Node.js. Diferente do pacote `sqlite3` tradicional, ele opera de forma **síncrona**, eliminando callbacks e permitindo que o aluno se concentre no SQL puro.
- `express`: Versão 5.x. Um detalhe vital da versão 5 do Express: **suporte nativo a Promises rejeitadas em middlewares e rotas**. No Express 4, erros assíncronos não capturados travavam a aplicação a menos que você usasse `next(err)`. No Express 5, um `throw new Error()` é automaticamente encaminhado ao middleware de erros!
- `multer`: Middleware robusto para manuseio de dados multipart/form-data, utilizado especificamente para uploads de arquivos.
- `zod`: Biblioteca de declaração e validação de esquemas de dados com inferência estática de tipos TypeScript.
- `tsx`: Executor e observador (file watcher) ultrarrápido para TypeScript. Permite rodar arquivos `.ts` diretamente sem compilação prévia manual para `.js`.

2.2. O Arquivo `tsconfig.json`

O `tsconfig.json` configura o compilador oficial da Microsoft para o TypeScript:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "preserve",
    "moduleResolution": "bundler",
    "types": ["node"],
    "strict": true,
    "allowImportingTsExtensions": true,
    "noUncheckedIndexedAccess": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["src", "scripts"]
}
```

Por que essas opções foram escolhidas?

1. `"strict": true` : Ativa todas as verificações estritas do TypeScript (como não permitir `null` ou `undefined` implícitos). Evita o temido `TypeError: Cannot read properties of undefined` em tempo de execução.
2. `"allowImportingTsExtensions": true` : Permite que importemos arquivos com a extensão `.ts` explícita (ex.: `import { db } from "./db.ts"`). Essa é uma prática moderna em ambientes baseados em ESM e `tsx`.
3. `"noEmit": true` : O compilador TypeScript (`tsc`) será usado exclusivamente para checar tipos (`npm run check`), e não para gerar código JavaScript em disco, pois quem executa o código diretamente é o `tsx`.

CAPÍTULO 3: A CAMADA DE PERSISTÊNCIA & BANCO DE DADOS RELACIONAL

Em sistemas de saúde, prontuários eletrônicos exigem precisão cirúrgica no armazenamento. O modelo de dados do nosso Mini-Prontuário é inspirado no padrão internacional **HL7 FHIR (Fast Healthcare Interoperability Resources)**, utilizando as entidades `Patient` (Paciente) e `Encounter` (Atendimento/Encontro Clínico).

3.1. O Esquema do Banco: `database/schema.sql`

Vejamos o arquivo SQL que define a estrutura de dados:

```
DROP TABLE IF EXISTS encounters;
```

```
DROP TABLE IF EXISTS patients;
```

```
CREATE TABLE patients (
```

```
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  name        TEXT      NOT NULL,
```

```
  birth_date  TEXT      NOT NULL,          -- ISO 8601: AAAA-MM-DD
```

```
  national_id TEXT      NOT NULL UNIQUE,   -- Cartao Nacional de Saude (CNS)
```

```
  active      INTEGER NOT NULL DEFAULT 1,  -- SQLite nao tem BOOLEAN: 0 ou 1
```

```
  photo_path  TEXT              -- caminho da foto (ex.: /uploads/xxx.jpg)
```

```
);
```

```
CREATE TABLE encounters (
```

```
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  patient_id  INTEGER NOT NULL,
```

```
  started_at  TEXT      NOT NULL,          -- ISO 8601: AAAA-MM-DDTHH:MM
```

```
  chief_complaint TEXT    NOT NULL,        -- queixa principal
```

```
  notes       TEXT,                -- conduta (opcional)
```

```
  FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
```

```
);
```

```
CREATE INDEX idx_encounters_patient ON encounters(patient_id);
```

Análise Conceitual e Didática do SQL:

1. `DROP TABLE IF EXISTS ...` na ordem correta:

Observe que `encounters` é deletada **antes** de `patients`. Por quê? Porque `encounters` depende de `patients` via chave estrangeira! Tentar deletar uma tabela pai antes da tabela filha violaria a integridade referencial.

2. **Datas como TEXT no formato ISO 8601:**

O SQLite não possui um tipo de dados `DATETIME` nativo como o PostgreSQL ou MySQL. Ele

```
AAAA-MM-DDTHH:MM, ORDER BY started_at DESC,
```

funcionem perfeitamente por ordem alfabética cronológica.

3. `national_id TEXT NOT NULL UNIQUE`:

O Cartão Nacional de Saúde (CNS) é um identificador único de cada cidadão no SUS. O modificador `UNIQUE` cria automaticamente um índice B-Tree no banco de dados e impede que dois pacientes tenham o mesmo número. Se tentarmos inserir um duplicado, o banco dispara um erro de violação de unicidade.

4. `active INTEGER NOT NULL DEFAULT 1`:

O SQLite também não tem tipo `BOOLEAN`. A convenção universal é usar inteiros: `0` representa `falso` e `1` representa `verdadeiro`.

5. `FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE`:

Essa cláusula define que cada atendimento pertence obrigatoriamente a um paciente existente. Se o paciente for deletado, todos os seus atendimentos serão apagados automaticamente em cascata (`ON DELETE CASCADE`), evitando registros órfãos.

6. `CREATE INDEX idx_encounters_patient ON encounters(patient_id);`:

 **Dica do Professor:** Sem esse índice, toda vez que o sistema buscasse os atendimentos de um paciente (`WHERE patient_id = ?`), o SQLite precisaria fazer um *Full Table Scan* (varrer a tabela inteira do primeiro ao último registro). O índice cria uma estrutura em árvore que localiza os atendimentos em tempo logarítmico $O(\log n)$.

3.2. Carga Inicial de Testes: `database/seed.sql`

O arquivo `database/seed.sql` fornece uma base inicial com dados realistas (embora fictícios):

```

INSERT INTO patients (name, birth_date, national_id, active) VALUES
  ('Maria da Conceicao Silva', '1978-03-14', '7000000000000001', 1),
  ('Joao Pedro Alves Santos', '1990-07-22', '7000000000000002', 1),
  ('Francisca das Chagas Souza', '1965-11-02', '7000000000000003', 1),
  ('Antonio Carlos Ferreira', '2001-01-30', '7000000000000004', 0),
  ('Raimunda Nonata Costa', '1988-05-19', '7000000000000005', 1);

INSERT INTO encounters (patient_id, started_at, chief_complaint, notes) VALUES
  (1, '2026-08-03T08:15', 'Cefaleia ha tres dias', 'Orientada hidratacao. Retorno em 7 dias.'),
  (1, '2026-08-10T14:40', 'Retorno - cefaleia', 'Melhora do quadro. Alta.'),
  (2, '2026-07-29T10:05', 'Dor lombar apos esforco', 'Repouso relativo e analgesia.'),
  (3, '2026-08-11T09:00', 'Controle de pressao arterial', 'PA 130x80. Mantida a medicacao.'),
  (5, '2026-08-12T16:20', 'Tosse seca persistente', NULL);

```

Observe que os CNSs fictícios começam com `700...` e há variações importantes: pacientes inativos (`active = 0`) e atendimentos com `notes = NULL`, fundamentais para testar a robustez do frontend.

3.3. Automação de Reset: `scripts/reset-db.ts`

```

import { readFileSync } from "node:fs";
import { join } from "node:path";
import { db } from "../src/db/database.ts";

const schema = readFileSync(join(process.cwd(), "database", "schema.sql"), "utf-8");
const seed = readFileSync(join(process.cwd(), "database", "seed.sql"), "utf-8");

db.exec(schema);
db.exec(seed);

console.log("Banco recriado com sucesso: database/prontuario.db");

```

Esse script roda com `npm run db:reset`. Ele lê o SQL do disco e executa o lote no banco de dados via `db.exec()`. Se seus testes ficarem sujos ou corrompidos, em meio segundo seu ambiente volta ao estado limpo.

3.4. Conexão do Banco de Dados: `src/db/database.ts`

```
import Database from "better-sqlite3";
import { join } from "node:path";

export const DATABASE_FILE = join(process.cwd(), "database", "prontuario.db");

export const db = new Database(DATABASE_FILE);

// SQLite nao aplica chave estrangeira por padrao. Isso liga a verificacao.
db.pragma("foreign_keys = ON");
```

Ponto de Atenção Crucial:

Por razões históricas de compatibilidade com versões antigas dos anos 2000, o **SQLite vem com o suporte a chaves estrangeiras DESLIGADO por padrão.**

A linha `db.pragma("foreign_keys = ON");` é **obrigatória**. Sem ela, você poderia inserir um atendimento para um paciente com `patient_id = 999999` e o banco aceitaria sem chiar!

CAPÍTULO 4: A CAMADA DE NEGÓCIO: OS SERVICES

Chegamos ao coração da aplicação. Se o software fosse uma empresa, o Service seria o especialista técnico que toma decisões e realiza o trabalho pesado.

4.1. `src/services/patients.service.ts`

Vamos estudar o código completo e comentar cada seção:

```
import { db } from "../db/database.ts";
import { BadRequestError, ConflictError, NotFoundError } from "../errors/HttpError.ts";

export type PatientRow = {
  id: number;
  name: string;
  birth_date: string;
  national_id: string;
  active: number;
  photo_path: string | null;
};

export type Patient = {
  id: number;
  name: string;
  birthDate: string;
  nationalId: string;
  active: boolean;
  photoUrl: string | null;
};

export function toPatientJson(row: PatientRow): Patient {
  return {
    id: row.id,
    name: row.name,
    birthDate: row.birth_date,
    nationalId: row.national_id,
    active: row.active === 1,
    photoUrl: row.photo_path,
  };
}
```

O Padrão Data Mapper (`toPatientJson`):

Note a diferença vital entre `PatientRow` e `Patient` :

- No banco de dados relacional, a convenção universal é o uso de **snake_case** (`birth_date` , `national_id` , `photo_path`) e o booleano é `number` (`0` ou `1`).
- No mundo JavaScript/JSON, a convenção absoluta é **camelCase** (`birthDate` , `nationalId` , `photoUrl`) e booleanos são `true` ou `false` .

A função `toPatientJson` atua como um **Mapper (Tradutor de Formato)**. Ela isola o esquema do banco de dados do contrato público da API. Se amanhã o nome da coluna no banco mudar de `photo_path` para `avatar_file` , só alteramos o mapper, e o frontend nem percebe!

As Operações do Service:

```

export const patientsService = {
  // 1. Listar todos os pacientes ordenados por nome
  list(): Patient[] {
    const rows = db
      .prepare("SELECT id, name, birth_date, national_id, active, photo_path FROM
patients ORDER BY name")
      .all() as PatientRow[];

    return rows.map(toPatientJson);
  },

  // 2. Buscar paciente por ID
  getById(id: string | number): Patient {
    const row = db
      .prepare("SELECT id, name, birth_date, national_id, active, photo_path FROM
patients WHERE id = ?")
      .get(id) as PatientRow | undefined;

    if (!row) {
      throw new NotFoundError("Paciente nao encontrado.");
    }

    return toPatientJson(row);
  },

  // 3. Cadastrar paciente com checagem de duplicidade
  create(data: { name: string; birthDate: string; nationalId: string }): Patient {
    const problem = validatePatientInput(data);
    if (problem) {
      throw new BadRequestError(problem);
    }

    const { name, birthDate, nationalId } = data;

    const duplicate = db
      .prepare("SELECT id FROM patients WHERE national_id = ?")
      .get(nationalId.trim());

    if (duplicate) {
      throw new ConflictError("Ja existe um paciente com este CNS.");
    }

    const result = db
      .prepare(
        `INSERT INTO patients (name, birth_date, national_id, active)
VALUES (?, ?, ?, 1)`
      )
      .run(name.trim(), birthDate, nationalId.trim());

    const created = db
      .prepare("SELECT id, name, birth_date, national_id, active, photo_path FROM
patients WHERE id = ?")
      .get(result.lastInsertRowid) as PatientRow;

    return toPatientJson(created);
  },

  // 4. Vincular foto de upload ao paciente
  setPhoto(id: string | number, filename: string): Patient {
    // Garante que o paciente existe (dispara NotFoundError se nao existir)
    patientsService.getById(id);

    const photoPath = `/uploads/${filename}`;
    db.prepare("UPDATE patients SET photo_path = ? WHERE id = ?").run(photoPath, id);

    const updated = db
      .prepare("SELECT id, name, birth_date, national_id, active, photo_path FROM
patients WHERE id = ?")
      .get(id) as PatientRow;

    return toPatientJson(updated);
  },
};

```

Análise das Boas Práticas Implementadas:

1. Uso de Prepared Statements (`db.prepare(...)`):

Ao usar o ponto de interrogação `?` e passar os valores nos métodos `.get()`, `.all()` ou `.run()`, o driver trata as variáveis como dados puros, e **não** como comandos executáveis. Isso elimina 100% da vulnerabilidade de **SQL Injection**.

2. Reuso de Lógica com `getById` :

Em `setPhoto`, antes de executar o `UPDATE`, chamamos `patientsService.getById(id)`. Se o paciente não existir, a exceção `NotFoundError` é lançada imediatamente, interrompendo a execução de forma limpa.

3. Lançamento de Erros Semânticos:

O service não faz `res.status(409)`. Ele simplesmente lança `throw new ConflictError(...)`. A responsabilidade de saber qual código HTTP devolver pertence ao middleware central de erros.

4.2. `src/services/encounters.service.ts`

Este service lida com os atendimentos clínicos:

```

import { db } from "../db/database.ts";
import { BadRequestError, NotFoundError } from "../errors/HttpError.ts";

export type EncounterRow = {
  id: number;
  patient_id: number;
  started_at: string;
  chief_complaint: string;
  notes: string | null;
};

export type Encounter = {
  id: number;
  patientId: number;
  startedAt: string;
  chiefComplaint: string;
  notes: string | null;
};

export function toEncounterJson(row: EncounterRow): Encounter {
  return {
    id: row.id,
    patientId: row.patient_id,
    startedAt: row.started_at,
    chiefComplaint: row.chief_complaint,
    notes: row.notes,
  };
}

function patientExists(id: string): boolean {
  return db.prepare("SELECT 1 FROM patients WHERE id = ?").get(id) !== undefined;
}

export const encountersService = {
  list(patientId: string): Encounter[] {
    if (!patientExists(patientId)) {
      throw new NotFoundError("Paciente nao encontrado.");
    }

    const rows = db
      .prepare(
        `SELECT id, patient_id, started_at, chief_complaint, notes
        FROM encounters
        WHERE patient_id = ?
        ORDER BY started_at DESC`
      )
      .all(patientId) as EncounterRow[];

    return rows.map(toEncounterJson);
  },

  create(patientId: string, data: { startedAt: string; chiefComplaint: string; notes?: string | null }): Encounter {
    if (!patientExists(patientId)) {
      throw new NotFoundError("Paciente nao encontrado.");
    }

    const { startedAt, chiefComplaint, notes } = data ?? {};

    if (isBlank(chiefComplaint)) {
      throw new BadRequestError("O campo 'chiefComplaint' e obrigatorio.");
    }

    if (isBlank(startedAt) || !ISO_DATE_TIME.test(startedAt)) {
      throw new BadRequestError("O campo 'startedAt' e obrigatorio no formato AAAA-MM-DDTHH:MM.");
    }

    const result = db
      .prepare(
        `INSERT INTO encounters (patient_id, started_at, chief_complaint, notes)
        VALUES (?, ?, ?, ?)`
      )
      .run(patientId, startedAt, chiefComplaint.trim(), isBlank(notes) ? null : notes!.trim());

    const created = db
      .prepare(
        `SELECT id, patient_id, started_at, chief_complaint, notes
        FROM encounters WHERE id = ?`
      )
      .get(result.lastInsertRowid) as EncounterRow;

    return toEncounterJson(created);
  },
};

```

O Truque de Performance: `SELECT 1 FROM patients`

Note a função auxiliar:

```
function patientExists(id: string): boolean {  
    return db.prepare("SELECT 1 FROM patients WHERE id = ?").get(id) !== undefined;  
}
```

Em vez de fazer `SELECT * FROM patients WHERE id = ?` (o que carregaria todas as colunas desnecessariamente para a memória), fazemos `SELECT 1`. O banco de dados para a busca assim que encontrar o primeiro registro correspondente no índice da chave primária, consumindo o mínimo de recursos.

CAPÍTULO 5: A CAMADA DE TRADUÇÃO & ORQUESTRAÇÃO: OS CONTROLLERS

Se o Service é o especialista de negócios e o banco de dados é o arquivo, o **Controller é o recepcionista bilíngue**. Ele fala fluentemente "HTTP" (com requisições, headers, status codes e JSON) e fala fluentemente a "linguagem do Domínio".

5.1. `src/controllers/patients.controller.ts`

```
import type { Request, Response } from "express";
import { patientsService } from "../services/patients.service.ts";
import { UnprocessableEntityError } from "../errors/HttpError.ts";

export const patientsController = {
  list(_req: Request, res: Response) {
    const patients = patientsService.list();
    res.status(200).json(patients);
  },

  getById(req: Request, res: Response) {
    const patient = patientsService.getById(req.params.id as string);
    res.status(200).json(patient);
  },

  create(req: Request, res: Response) {
    const created = patientsService.create(req.body);
    res.status(201).json(created);
  },

  uploadPhoto(req: Request, res: Response) {
    if (!req.file) {
      throw new UnprocessableEntityError("Arquivo de foto nao enviado.");
    }
    const patient = patientsService.setPhoto(req.params.id as string, req.file.filename);
    res.status(200).json(patient);
  },
};
```

O que o Controller Faz e o que ele NUNCA Faz:

• Ele FAZ:

- ° Extrai os parâmetros das requisições: `req.params.id`, `req.body`, `req.file`.
- ° Passa esses dados para o Service.
- ° Define o código HTTP apropriado: `200 OK` para consultas e atualizações, `201 Created` para inserções bem-sucedidas.
- ° Devolve o resultado em formato JSON com `res.json(...)`.

• Ele NUNCA FAZ:

- ° NUNCA executa SQL (`db.prepare`).
- ° NUNCA faz regras como "verificar se CNS já existe".
- ° NUNCA faz `try/catch` para capturar erros e formatar respostas manuais de erro. Se o Service lançar uma exceção, o Controller deixa ela subir, e o Express a entrega diretamente para o `errorHandler`.

5.2. `src/controllers/encounters.controller.ts`

```
import type { Request, Response } from "express";
import { encountersService } from "../services/encounters.service.ts";

export const encountersController = {
  list(req: Request, res: Response) {
    const patientId = req.params.id as string;
    const encounters = encountersService.list(patientId);
    res.status(200).json(encounters);
  },

  create(req: Request, res: Response) {
    const patientId = req.params.id as string;
    const created = encountersService.create(patientId, req.body);
    res.status(201).json(created);
  },
};
```

Observe a simplicidade e a beleza visual de um Controller enxuto. Cada método possui entre 3 e 4 linhas de código. O código é autoexplicativo, de fácil leitura e manutenção.

CAPÍTULO 6: A CAMADA DE ENTRADA: AS ROTAS MODULARES

As rotas são a "porta de entrada" da aplicação. Elas definem quais URLs o servidor escuta e conectam essas URLs aos middlewares e aos controllers correspondentes.

6.1. `src/routes/patients.routes.ts`

```
import { Router } from "express";
import { patientsController } from "../controllers/patients.controller.ts";
import { validate } from "../middlewares/validate.ts";
import { createPatientSchema } from "../validation/patients.schemas.ts";
import { uploadPhoto } from "../middlewares/upload.ts";

export const patientsRouter = Router();

patientsRouter.get("/", patientsController.list);
patientsRouter.get("/:id", patientsController.getById);
patientsRouter.post("/", validate(createPatientSchema), patientsController.create);
patientsRouter.post("/:id/photo", uploadPhoto.single("photo"),
patientsController.uploadPhoto);
```

Composição em Pipeline:

Preste atenção na linha do `POST / :`

```
patientsRouter.post("/", validate(createPatientSchema), patientsController.create);
```

Isso é um **Pipeline de Execução**. Quando uma requisição chega:

1. Primeiro ela passa pelo middleware `validate(createPatientSchema)`. Se o JSON for inválido, uma exceção é lançada e o pipeline é interrompido imediatamente.
2. Apenas se os dados forem 100% válidos, a execução segue para `patientsController.create`.

Na rota de upload de foto:

```
patientsRouter.post("/:id/photo", uploadPhoto.single("photo"),
patientsController.uploadPhoto);
```

1. Primeiro o middleware `uploadPhoto.single("photo")` intercepta o fluxo multipart, processa o arquivo, salva em disco e anexa o objeto em `req.file`.
2. Em seguida, `patientsController.uploadPhoto` pega o nome do arquivo gerado e associa ao paciente.

6.2. `src/routes/encounters.routes.ts`

```
import { Router } from "express";
import { encountersController } from "../controllers/encounters.controller.ts";

export const encountersRouter = Router({ mergeParams: true });

encountersRouter.get("/", encountersController.list);
encountersRouter.post("/", encountersController.create);
```

⚠ A Pegadinha de Prova: O Parâmetro `{ mergeParams: true }`

Este é um dos tópicos mais cobrados em avaliações e entrevistas sobre Express!

No arquivo `server.ts`, nós montamos esse roteador da seguinte forma:

```
app.use("/api/patients/:id/encounters", encountersRouter);
```

Observe que o parâmetro `:id` foi capturado no caminho **pai**.

Por padrão, cada instância de `Router()` do Express possui seu próprio escopo isolado de parâmetros. Sem a opção `{ mergeParams: true }`, o valor de `:id` seria **perdido** e chegaria como `undefined` dentro do controller de atendimentos!

Ao passar `{ mergeParams: true }`, dizemos ao Express: *"Por favor, propague os parâmetros da rota pai para dentro deste roteador filho"*.

CAPÍTULO 7: TRATAMENTO ROBUSTO DE ERROS: HIERARQUIA E MIDDLEWARE CENTRAL

7.1. O Anti-padrão dos Erros Espalhados

Em códigos amadores, é comum encontrar estruturas como esta repetidas dezenas de vezes:

```
// ❌ CÓDIGO RUIM - NÃO FAÇA ISSO!  
try {  
  // ...  
} catch (err) {  
  res.status(500).json({ error: "Erro interno no servidor" });  
}
```

Isso gera:

- * Respostas inconsistentes (às vezes `{ error: "msg" }`, às vezes `{ message: "msg" }`, às vezes texto puro);
- * Vazamento acidental de stack traces internos do banco de dados para o cliente;
- * Falta de logs padronizados.

7.2. A Hierarquia Polimórfica: `src/errors/HttpError.ts`

Criamos uma hierarquia elegante orientada a objetos herdando da classe nativa `Error` do JavaScript:

```
export class HttpError extends Error {
  constructor(
    public statusCode: number,
    message: string,
    public details?: unknown
  ) {
    super(message);
    this.name = this.constructor.name;
  }
}

export class BadRequestError extends HttpError {
  constructor(message = "Requisição inválida", details?: unknown) {
    super(400, message, details);
  }
}

export class NotFoundError extends HttpError {
  constructor(message = "Recurso não encontrado") {
    super(404, message);
  }
}

export class ConflictError extends HttpError {
  constructor(message = "Conflito com o estado atual") {
    super(409, message);
  }
}

export class UnprocessableEntityError extends HttpError {
  constructor(message = "Não foi possível processar", details?: unknown) {
    super(422, message, details);
  }
}

export class PayloadTooLargeError extends HttpError {
  constructor(message = "Arquivo excede o tamanho permitido") {
    super(413, message);
  }
}
```

Por que isso é brilhante?

1. **Semântica:** Em qualquer lugar do sistema, para avisar que um recurso não existe, basta escrever:
`throw new NotFoundError("Paciente não encontrado.")`.
 2. **Polimorfismo:** Qualquer erro criado é uma instância de `HttpError`, permitindo que o middleware capture todos com um simples `err instanceof HttpError`.
 3. **TypeScript First-Class:** O uso de `public statusCode: number` no construtor utiliza o recurso *Parameter Properties* do TypeScript, declarando e atribuindo a propriedade automaticamente.
-

7.3. O Middleware Central: `src/middlewares/errorHandler.ts`

No Express, um middleware de erro é reconhecido pela sua assinatura especial contendo **exatamente 4 argumentos**: `(err, req, res, next)`.

```

import type { Request, Response, NextFunction } from "express";
import multer from "multer";
import { HttpError } from "../errors/HttpError.ts";

export function errorHandler(
  err: unknown,
  _req: Request,
  res: Response,
  _next: NextFunction
): void {
  // 1. Erro da nossa hierarquia customizada
  if (err instanceof HttpError) {
    res.status(err.statusCode).json({
      error: {
        message: err.message,
        statusCode: err.statusCode,
        details: err.details,
      },
    });
    return;
  }

  // 2. Erros originados pelo Multer (Upload)
  if (err instanceof multer.MulterError) {
    if (err.code === "LIMIT_FILE_SIZE") {
      res.status(413).json({
        error: {
          message: "Arquivo excede o tamanho permitido",
          statusCode: 413,
        },
      });
      return;
    }
    res.status(400).json({
      error: {
        message: err.message,
        statusCode: 400,
      },
    });
    return;
  }

  // 3. Erros inesperados de infraestrutura (Fallback 500)
  console.error(err);
  res.status(500).json({
    error: {
      message: "Erro interno do servidor",
      statusCode: 500,
    },
  });
}

```

O Contrato Único de Resposta:

A partir de agora, **100% dos erros** da nossa API seguem rigorosamente a mesma estrutura JSON:

```
{
  "error": {
    "message": "Descrição amigável do erro",
    "statusCode": 400,
    "details": { ... }
  }
}
```

Isso simplifica drasticamente a vida dos desenvolvedores frontend, que sabem exatamente como ler e exibir qualquer mensagem de erro que ocorra no sistema.

CAPÍTULO 8: INTEGRIDADE DE DADOS: VALIDAÇÃO DECLARATIVA COM ZOD

8.1. O que é o Zod?

Zod é uma biblioteca TypeScript-first para definição e validação de esquemas. Em vez de escrever dezenas de `if (typeof x !== 'string' || x.length < 1)` manuais, declaramos a forma esperada dos dados em um esquema conciso.

8.2. O Esquema do Paciente:

`src/validation/patients.schemas.ts`

```
import { z } from "zod";

const ISO_DATE = /^\\d{4}-\\d{2}-\\d{2}$/;

export const createPatientSchema = z.object({
  name: z
    .string({ required_error: "'Nome' é um campo obrigatorio e nao pode ser vazio." })
    .trim()
    .min(1, "'Nome' é um campo obrigatorio e nao pode ser vazio."),
  birthDate: z
    .string({ required_error: "'Data de nascimento' é um campo obrigatorio e deve estar no formato AAAA-MM-DD." })
    .regex(ISO_DATE, "'Data de nascimento' é um campo obrigatorio e deve estar no formato AAAA-MM-DD."),
  nationalId: z
    .string({ required_error: "'CNS' é um campo obrigatorio." })
    .trim()
    .min(1, "'CNS' é um campo obrigatorio."),
});

export type CreatePatientInput = z.infer<typeof createPatientSchema>;
```

Recursos Avançados Utilizados Aqui:

1. `.trim()` : Remove automaticamente espaços em branco no início e fim das strings antes de validar o comprimento. Evita que o usuário envie " " e passe batido.
2. `YYYY-MM-DD`
3. `z.infer<typeof createPatientSchema>` : **Magia do TypeScript!** Não precisamos criar uma interface manual duplicando os campos. O Zod gera o tipo TypeScript estático automaticamente a partir do esquema de validação em tempo de execução!

8.3. O Middleware de Validação Genérico:

`src/middlewares/validate.ts`

```
import type { Request, Response, NextFunction } from "express";
import type { ZodSchema } from "zod";
import { BadRequestError } from "../errors/HttpError.ts";

export function validate(schema: ZodSchema) {
  return (req: Request, _res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      throw new BadRequestError("Dados inválidos", result.error.flatten().fieldErrors);
    }
    // Atribui os dados limpos e parseados de volta para o corpo da requisicao
    req.body = result.data;
    next();
  };
}
```

O Padrão Higher-Order Function (Factory de Middlewares):

A função `validate` recebe um esquema Zod qualquer e retorna uma função middleware do Express.

- Se a validação falhar (`!result.success`), ela extrai os erros por campo com `result.error.flatten().fieldErrors` e lança um `BadRequestError (400)` .
- Se tiver sucesso, ela atualiza `req.body = result.data` (o que descarta propriedades maliciosas que o usuário tenha tentado injetar no JSON) e chama `next()` .

CAPÍTULO 9: UPLOAD SEGURO DE ARQUIVOS COM MULTER

Permitir que usuários façam upload de arquivos para um servidor web é uma das operações de maior risco em segurança da informação.

Os 3 Grandes Riscos no Upload de Arquivos:

1. **Path Traversal:** O invasor envia um arquivo com o nome `../../../../../../etc/passwd` ou `../../../../server.ts` para sobrescrever arquivos vitais do sistema operacional.
2. **Denial of Service (DoS):** O cliente envia um arquivo de 50 Gigabytes, esgotando o armazenamento em disco e a memória RAM do servidor.
3. **MIME Type Spoofing / Execução Remota de Código (RCE):** O invasor envia um script malicioso renomeado para `.jpg` para tentar executá-lo no servidor.

9.1. A Configuração Segura: `src/middlewares/upload.ts`

```
import multer from "multer";
import path from "path";
import crypto from "crypto";
import { UnprocessableEntityError } from "../errors/HttpError.ts";

const storage = multer.diskStorage({
  destination: "uploads/",
  filename: (_req, file, cb) => {
    // Gera um UUID unico e concatena com a extensao original segura
    const uniqueName = `${crypto.randomUUID()}${path.extname(file.originalname)}`;
    cb(null, uniqueName);
  },
});

const ALLOWED = ["image/jpeg", "image/png"];

export const uploadPhoto = multer({
  storage,
  limits: { fileSize: 2 * 1024 * 1024 }, // Limite rigoroso de 2MB
  fileFilter: (_req, file, cb) => {
    if (!ALLOWED.includes(file.mimetype)) {
      return cb(new UnprocessableEntityError("Formato de arquivo invalido. Apenas JPEG e PNG sao permitidos."));
    }
    cb(null, true);
  },
});
```

Como essa configuração blindar nossa aplicação:

1. `crypto.randomUUID()` : O nome original do arquivo é descartado sumariamente. O arquivo recebe `e3b0c442-98fc-1c14-9af3-4c56e7890123.jpg`, **aniquila qualquer tentativa de Path Traversal**.
2. `limits: { fileSize: 2 * 1024 * 1024 }` : O Multer interrompe o fluxo de dados se o tamanho ultrapassar 2 Megabytes e emite o erro `LIMIT_FILE_SIZE`, capturado pelo nosso `errorHandler` com status `413 Payload Too Large`.
3. `fileFilter` : Apenas imagens `image/jpeg` e `image/png` são aceitas. Qualquer outro formato (como `.exe`, `.sh`, `.php`, `.pdf`) é rejeitado com status `422 Unprocessable Entity`.

CAPÍTULO 10: A INTEGRAÇÃO COMPLETA: O NOVO `server.ts`

Agora que cada peça foi devidamente construída e lapidada em seu respectivo arquivo, veja a elegância e a clareza do nosso servidor principal:

```

import express from "express";
import { patientsRouter } from "../routes/patients.routes.ts";
import { encountersRouter } from "../routes/encounters.routes.ts";
import { errorHandler } from "../middlewares/errorHandler.ts";

const app = express();
const PORT = 3000;

// 1. Middlewares globais de processamento inicial
app.use(express.json());
app.use(express.static("public"));
app.use("/uploads", express.static("uploads")); // Servir fotos salvas

// 2. Health check (auditoria de saúde da aplicação)
app.get("/api/health", (_request, response) => {
  response.json({ status: "ok" });
});

// 3. Montagem dos Roteadores Modulares
app.use("/api/patients", patientsRouter);
app.use("/api/patients/:id/encounters", encountersRouter);

// 4. Middleware de Tratamento Centralizado de Erros (SEMPRE POR ÚLTIMO!)
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Mini-Prontuario no ar em http://localhost:${PORT}`);
});

```

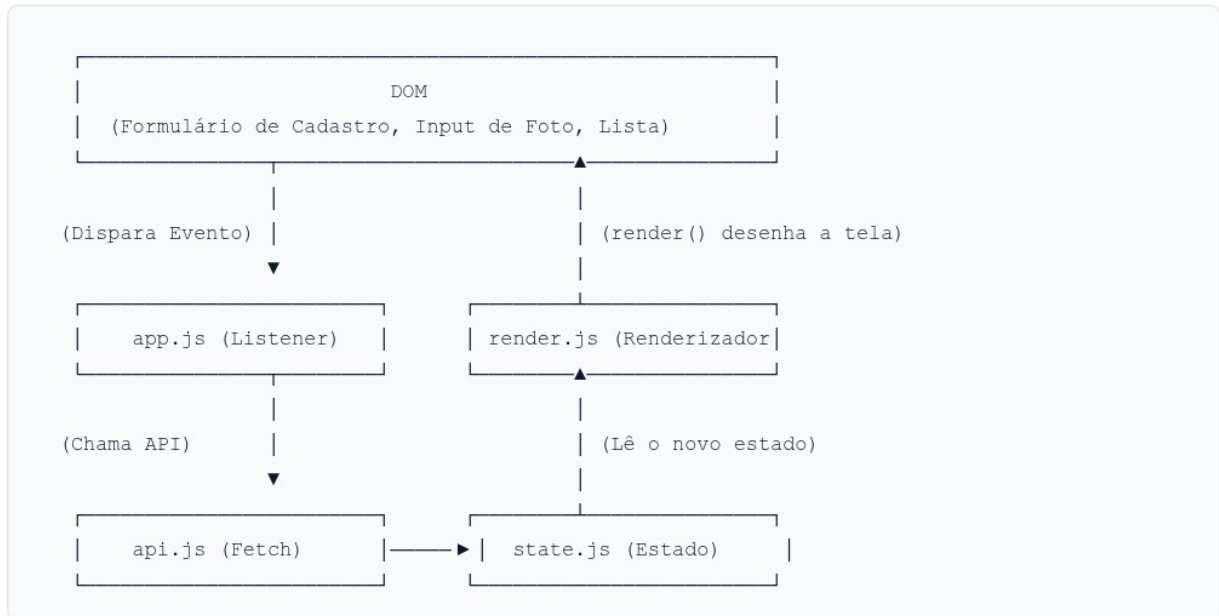
Por que a ordem dos middlewares importa tanto no Express?

O Express opera sob o padrão arquitetural **Chain of Responsibility**.

1. Se você registrar `app.use(errorHandler)` **antes** das rotas, o Express nunca o chamará quando ocorrer um erro na rota, pois ele não saberá que o manipulador já passou.
2. Se você registrar `express.static("public")` depois de uma rota curinga, os arquivos CSS e HTML não serão entregues.
3. A regra de ouro é:
 - **Primeiro:** Parsers de corpo (`express.json`) e pastas estáticas;
 - **Segundo:** Rotas e controladores da API;
 - **Por Último:** O Middleware Central de Erro.

CAPÍTULO 11: O FRONTEND MODULAR E REATIVO (VANILLA JAVASCRIPT)

O frontend do Mini-Prontuário foi construído com base em princípios modernos de engenharia web: **Arquitetura Unidirecional de Dados (estilo React/Redux)**, mas utilizando apenas **Vanilla JavaScript nativo**.



11.1. Estrutura HTML: `public/index.html`

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Mini-Prontuário</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous" />
  <link rel="stylesheet" href="./css/tokens.css" />
  <link rel="stylesheet" href="./css/base.css" />
</head>
<body>
  <header class="app-header">
    <div class="container">
      <p class="app-header__eyebrow">IFPI · Programação para Internet II · Tópico 2</p>
      <h1 class="app-header__title">Mini-Prontuário</h1>
    </div>
  </header>

  <main class="container app-main">
    <div id="form-error-container"></div>

    <section aria-label="Novo paciente" class="mb-4">
      <form id="patient-form" class="row g-2">
        <div class="col-12 col-md-4">
          <input class="form-control" name="name" placeholder="Nome completo" required />
        </div>
        <div class="col-6 col-md-3">
          <input class="form-control" name="birthDate" type="date" required />
        </div>
        <div class="col-6 col-md-3">
          <input class="form-control" name="nationalId" placeholder="CNS" required />
        </div>
        <div class="col-12 col-md-2">
          <button class="btn btn-success w-100" type="submit">Cadastrar</button>
        </div>
      </form>
    </section>

    <ul id="patient-list" class="patient-list"></ul>
  </main>

  <script type="module" src="./js/app.js"></script>
</body>
</html>
```

```
<script type="module" src="./js/app.js">  
,      import      export      ,  
</script>
```

11.2. Design Tokens & CSS: `tokens.css` e `base.css`

Em `public/css/tokens.css`, definimos as variáveis globais de design:

```
:root {  
  --color-primary: #0b6b4f;  
  --color-primary-soft: #d6f5e9;  
  --color-danger: #a33518;  
  --color-danger-soft: #ffe4dc;  
  --color-text: #12151a;  
  --color-muted: #5f6b7a;  
  --color-border: #d9dee6;  
  --color-surface: #ffffff;  
  --color-bg: #f6f7f9;  
  --space-1: 4px;  
  --space-2: 8px;  
  --space-3: 16px;  
  --space-4: 24px;  
  --radius: 10px;  
}
```

Isso permite alterar o tema visual inteiro do sistema em um único lugar, mantendo harmonia cromática em toda a interface.

11.3. O Estado Central: `public/js/state.js`

```
export const state = {
  patients: [],
  formError: null,
  fieldErrors: {},
};

export function setPatients(patients) {
  state.patients = patients;
}

export function addPatient(patient) {
  state.patients = [...state.patients, patient].sort((a, b) => a.name.localeCompare(b.name));
}

export function setFormError(message, fieldErrors = {}) {
  state.formError = message;
  state.fieldErrors = fieldErrors;
}

export function clearFormError() {
  state.formError = null;
  state.fieldErrors = {};
}
```

Por que usar um estado centralizado?

Em aplicações web mal projetadas, as funções buscam valores diretamente no DOM com `document.querySelector`, alteram classes manualmente e perdem o controle de quais dados estão na tela.

Aqui, aplicamos o conceito de **Single Source of Truth (Fonte Única da Verdade)**: o DOM não armazena dados; ele é apenas o espelho reflexivo do objeto `state`.

11.4. Comunicação com a API: `public/js/api.js`

```
const PATIENTS_URL = "/api/patients";

export async function listPatients() {
  const response = await fetch(PATIENTS_URL);
  if (!response.ok) throw new Error(`Falha ao carregar pacientes (HTTP ${response.status})`);
  return response.json();
}

export async function createPatient(patient) {
  const response = await fetch(PATIENTS_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(patient),
  });
  const body = await response.json();
  if (!response.ok) throw { apiError: body };
  return body;
}

export async function uploadPatientPhoto(patientId, file) {
  const formData = new FormData();
  formData.append("photo", file);
  const response = await fetch(`${PATIENTS_URL}/${patientId}/photo`, {
    method: "POST",
    body: formData, // SEM Content-Type manual!
  });
  const body = await response.json();
  if (!response.ok) throw { apiError: body };
  return body;
}
```

Dica de Ouro sobre Uploads com `fetch`:

```
headers: { "Content-Type": "multipart/form-data" } // FormData fetch
```

Quando você passa um `FormData` no `body`, o próprio navegador define automaticamente o `Content-Type` com o delimitador correto (`boundary`), por exemplo:

```
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryXyZ123.
```

Se você colocar o header manualmente, o delimitador `boundary` não será enviado, e o servidor Express/Multer não conseguirá decodificar o arquivo!

11.5. Renderização Declarativa: `public/js/render.js`

```
import { state } from "../state.js";

export function render() {
  renderFormError();
  renderPatientList();
}

function renderFormError() {
  const container = document.getElementById("form-error-container");
  container.innerHTML = "";
  if (!state.formError) return;

  const box = document.createElement("div");
  box.className = "form-error";
  box.textContent = state.formError;
  container.appendChild(box);
}

function renderPatientList() {
  const list = document.getElementById("patient-list");
  list.innerHTML = "";

  for (const patient of state.patients) {
    const li = document.createElement("li");
    li.className = "patient-card";
    li.dataset.patientId = patient.id;

    const photoSrc = patient.photoUrl || "";
    li.innerHTML = `
      ${photoSrc ? `` : `<div
class="patient-card__photo"></div>`}
      <div>
        <p class="patient-card__name">${patient.name}</p>
        <p class="patient-card__meta">CNS <span data-pdf-math="1"></span>{patient.active ?
"ativo" : "inativo"}</p>
      </div>
    `;
    list.appendChild(li);
  }
}
```

A função `render()` é idenfotente: qualquer que seja o momento em que for chamada, ela lê o estado atual e desenha os componentes com perfeição.

11.6. Orquestração de Eventos: `public/js/app.js`

```
import { listPatients, createPatient } from "../api.js";
import { state, setPatients, addPatient, setFormError, clearFormError } from "../state.js";
import { render } from "../render.js";
import { renderApiError } from "../errors.js";

async function init() {
  try {
    const patients = await listPatients();
    setPatients(patients);
  } catch (err) {
    setFormError(err.message ?? "Falha ao carregar pacientes.");
  }
  render();
}

document.getElementById("patient-form").addEventListener("submit", async (event) => {
  event.preventDefault();
  clearFormError();

  const form = event.target;
  const payload = {
    name: form.name.value,
    birthDate: form.birthDate.value,
    nationalId: form.nationalId.value,
  };

  try {
    const created = await createPatient(payload);
    addPatient(created);
    form.reset();
  } catch (err) {
    if (err.apiError) {
      renderApiError(err.apiError);
    } else {
      setFormError(err.message ?? "Falha ao cadastrar paciente.");
      render();
    }
    return;
  }
  render();
});

init();
```

Ao carregar a página, `init()` busca os pacientes e atualiza a lista. Ao submeter o formulário, capturamos os dados, enviamos à API e, em caso de erro, acionamos `renderApiError` com o contrato consistente da API.

CAPÍTULO 12: LABORATÓRIO DE TESTES HTTP & AUDITORIA

O arquivo `requests.http` é o laboratório de testes do projeto. Ele permite rodar requisições diretamente de extensões como REST Client (VS Code) ou ferramentas similares.

Vamos estudar a anatomia dos testes e os comportamentos esperados:

Teste 1: Health Check (Status da Aplicação)

```
GET http://localhost:3000/api/health
```

* Resposta esperada: `200 OK`

```
{ "status": "ok" }
```

Teste 2: Listar Pacientes

```
GET http://localhost:3000/api/patients
```

* Resposta esperada: `200 OK` com array de pacientes ordenados por nome em camelCase.

Teste 3: Falha Proposital - Paciente Inexistente (404)

```
GET http://localhost:3000/api/patients/9999
```

* Resposta esperada: `404 Not Found`

```
{
  "error": {
    "message": "Paciente nao encontrado.",
    "statusCode": 404
  }
}
```

Teste 4: Falha Proposital - Conflito de CNS Duplicado (409)

```
POST http://localhost:3000/api/patients
Content-Type: application/json

{
  "name": "Paciente Duplicado",
  "birthDate": "1990-01-01",
  "nationalId": "7000000000000001"
}
```

• Resposta esperada: 409 Conflict

```
{
  "error": {
    "message": "Ja existe um paciente com este CNS.",
    "statusCode": 409
  }
}
```

Teste 5: Falha Proposital - Validação Zod com Detalhes (400)

```
POST http://localhost:3000/api/patients
Content-Type: application/json

{
  "name": "",
  "birthDate": "data-invalida",
  "nationalId": ""
}
```

• Resposta esperada: 400 Bad Request com o objeto details :

```
{
  "error": {
    "message": "Dados inválidos",
    "statusCode": 400,
    "details": {
      "name": ["'Nome' é um campo obrigatorio e nao pode ser vazio."],
      "birthDate": ["'Data de nascimento' é um campo obrigatorio e deve estar no formato AAAA-MM-DD."],
      "nationalId": ["'CNS' é um campo obrigatorio."]
    }
  }
}
```

Teste 6: Upload de Foto Válida (200)

```
POST http://localhost:3000/api/patients/1/photo
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="photo"; filename="foto.jpg"
Content-Type: image/jpeg

< ./foto.jpg
-----WebKitFormBoundary--
```

- `"photoUrl":
"/uploads/uuid-gerado.jpg"`

CAPÍTULO 13: GUIA DE BOLSO DO DESENVOLVEDOR & CHECKLIST DE PROVA

Para encerrar este livro com chave de ouro, aqui está o seu guia rápido para consulta em provas práticas, revisões de código e projetos profissionais.

13.1. Tabela Periódica dos Códigos de Status HTTP

Código	Nome	Quando Usar	Exemplo no Projeto
200	OK	Sucesso em requisições de consulta (<code>GET</code>) ou atualização (<code>PUT/PATCH</code>).	<code>GET /api/patients</code>
201	Created	Sucesso na criação de um novo recurso (<code>POST</code>).	<code>POST /api/patients</code>
400	Bad Request	Dados de entrada malformados ou sintaticamente incorretos.	Validação Zod reprovada.
404	Not Found	O recurso solicitado não existe no banco de dados.	<code>GET /api/patients/9999</code>
409	Conflict	A requisição faz sentido, mas colide com o estado atual do banco.	Tentativa de cadastrar CNS duplicado.
413	Payload Too Large	Arquivo enviado ultrapassa o limite de tamanho permitido.	Upload de imagem maior que 2MB.
422	Unprocessable Entity	Dados sintaticamente válidos, mas semanticamente inaceitáveis.	Upload de arquivo que não seja JPEG/PNG.
500	Internal Server Error	Falha inesperada no servidor ou queda de infraestrutura.	Exceção não tratada.

13.2. As 7 Armadilhas Mais Comuns

1. **Esquecer o `mergeParams: true`**: O controller de atendimentos recebe `req.params.id` como `undefined`.
 2. **Esquecer de ligar o `foreign_keys = ON` no SQLite**: O banco permite inserir filhos órfãos sem chave estrangeira.
 3. **Fazer `res.status().json()` dentro do Service**: Quebra brutal da separação de camadas. O Service não deve saber o que é HTTP.
 4. **Colocar `app.use(errorHandler)` antes das rotas**: Erros não serão capturados e a aplicação travará ou devolverá página HTML padrão do Express.
 5. **Passar `Content-Type: multipart/form-data` no header do fetch manualmente**: O navegador perde o `boundary` e o Multer recusa o upload.
 6. **Armazenar fotos no banco como BLOB**: Sobrecarrega a memória do banco de dados relacional. A prática correta é salvar o arquivo no disco (ou bucket S3) e gravar no banco apenas o caminho/URL da imagem.
 7. **Usar o nome original do arquivo no upload**: Brecha grave para invasão por Path Traversal. Sempre gere um nome único com `crypto.randomUUID()`.
-

13.3. Mensagem Final do Professor

Parabéns por concluir este estudo!

A capacidade de pegar um sistema confuso e transformá-lo em uma arquitetura modular, testável e robusta é exatamente o que separa programadores amadores de verdadeiros engenheiros de software. Guarde este material, consulte-o sempre que for projetar novas APIs e bons estudos no IFPI!

[FIM DO EBOOK]

Mini-Prontuário — Do Monólito à Arquitetura em Camadas

Instituto Federal do Piauí (IFPI) · 2026